

QUESTÃO 1 - CÓDIGO ALUNO LIVIO SOUSA

```
#include<stdio.h>

int main(){
unsigned char r, g, b, m, b2, b1, b0;

puts("Entre com os valores de r, g e b:");
scanf("%hhu %hhu %hhu", &r, &g, &b);

puts("Entre com o valor de M");
scanf("%hhu", &m);

b2 = m>>2 & 1;
b1 = m>>1 & 1;
b0 = m & 1;

r = r&254 | b2;
g = g&254 | b1;
b = b&254 | b0;
printf("%hhu %hhu %hhu", r, g, b);

return 0;
}
```

QUESTÃO 1 - CÓDIGO CLAUDE

```
#include <stdio.h>
#include <stdint.h>

/* Substitui o bit menos significativo
de 'value' pelo bit fornecido (0 ou 1). */
static uint8_t set_lsb(uint8_t value,
uint8_t bit) {
return (uint8_t)((value & 0xFEu) | (bit &
0x01u));
}

int main(void) {
uint8_t r, g, b, m;

printf("Digite R G B (0-255 cada): ");
if (scanf("%hhu %hhu %hhu", &r, &g,
&b) != 3) {
fprintf(stderr, "Entrada invalida para R,
G, B.\n");
return 1;
}
printf("Digite M (0-7): ");
if (scanf("%hhu", &m) != 1) {
fprintf(stderr, "Entrada invalida para M.
\n");
return 1;
}

r = set_lsb(r, (m >> 2) & 1u);
g = set_lsb(g, (m >> 1) & 1u);
b = set_lsb(b, m & 1u);

printf("R = %hhu\n", r);
printf("G = %hhu\n", g);
printf("B = %hhu\n", b);
return 0;
}
```

Questão 1:

- i) O Claude atribuiu às variáveis r, g, b e m o tipo int modificado para 8 bits(em vez de 32 bits originalmente) alegando melhor portabilidade, pois em algumas plataformas exóticas, unsigned char pode ter mais de 8 bits.
- ii) As operações de limpeza e enxerto de bit foram feitas via função alegando manutenção mais fácil. Concordo, o uso de funções deve ser encorajado. Essa foi a alteração mais relevante.
- iii) A quantidade de entradas(3 - R, G, B) foi checada antes da execução das operações principais. Isso parece ser uma boa prática, pois se o usuário deixar de digitar alguma entrada o programa vai rodar "normalmente" com lixo de memória, podendo até mesmo passar despercebido.
- iv) Os parênteses nas operações parecem ser uma boa prática, assim como o uso das máscaras em hexadecimal, que deixam explícito que aquele número trata-se de uma máscara. Vou tentar observar isso nos próximos códigos.
- iv) A impressão em três linhas parece ser irrelevante.

QUESTÃO 2 - CÓDIGO ALUNO LIVIO SOUSA

```
#include<stdio.h>

int main(){
float y;
unsigned int n, i, x1, x2, x3;
puts("Quantas amostras do sinal serao usadas?(minimo de 3 amostras)");
scanf("%u", &n);
if(n >= 3){
puts("Entre com x1 e x2");
scanf("%u %u", &x1, &x2);
for( i = 2; i <= (n - 1); ++i){
printf("Entre com x%u:\n", i+1);
scanf("%u", &x3);
y = (float)(x1 + x2+ x3)/3;
printf("Y%u = %.2f\n", i, y);
x1 = x2;
x2 = x3;
}
}else{
puts("Voce deve digitar no minimo 3 amostras. Entre novamente com a quantidade de amostras");
}
return 0;
}
```

QUESTÃO 2 - CÓDIGO CLAUDE

```
#include <stdio.h>

int main(void) {
unsigned long n, i;
double x_prev, x_curr, x_next;
printf("Quantidade de amostras (n >= 3):
");
if (scanf("%lu", &n) != 1) {
fprintf(stderr, "Entrada invalida.\n");
return 1;
}
if (n < 3) {
printf("E necessario informar pelo menos
3 amostras.\n");
return 0;
}
printf("Digite x1 e x2: ");
if (scanf("%lf %lf", &x_prev, &x_curr) != 2) {
fprintf(stderr, "Entrada invalida.\n");
return 1;
}
for (i = 2; i <= n - 1; i++) {
printf("Digite x%lu: ", i + 1);
if (scanf("%lf", &x_next) != 1) {
fprintf(stderr, "Entrada invalida.\n");
return 1;
}
printf("y%lu = %.4f\n", i, (x_prev + x_curr +
x_next) / 3.0);

x_prev = x_curr;
x_curr = x_next;
}
return 0;
}
```

- i) O Claude elevou as representações dos números alterando os tipos das variáveis. Essa alteração é cabível caso o processamento de sinais trabalhe com números altos e negativos. Para efeitos didáticos não há tanta relevância.
- ii) Em todas as leituras de dados há validação da quantidade de entradas, o que parece ser uma boa prática;
- iii) No meu código, o `for` fica dentro do desvio do `if`, que não é desejável. Essa observação é relevante e alterou meu modo de pensar na elaboração dos próximos algoritmos.
- iv) O código do Claude calcula os valores de `Y` dentro do `print`, sem necessidade de criação da variável `Y`, o que é recomendado como uma boa prática. Essa observação é relevante e alterou meu modo de pensar na elaboração dos próximos algoritmos.

QUESTÃO 3 - CÓDIGO ALUNO LIVIO SOUSA

```
#include<stdio.h>
int main(){
    unsigned int n;
    unsigned char x_int, min = 255, max = 0,
    x;
    float x_norm;
    puts("Quantos pixels tem a imagem?");
    scanf("%u", &n);
    for(unsigned int i = 1; i<= n; ++i){
        printf("Entre com a intensidade x%u de 0
a 255:", i);
        scanf("%hhu", &x_int);
        if(x_int < min){
            min = x_int;
        }
        if(x_int > max){
            max = x_int;
        }
    }
    if (min == max){
        puts("As intensidades dos pixels sao
todas iguais, nao sera possivel
normalizar x.");
    }
    else{
        printf("Entre com a intensidade x entre
%hhu e %hhu que sera normalizada:",
min, max);
        scanf("%hhu", &x);
        x_norm = (float)(x - min)/(max - min);
        printf("O valor de %hhu normalizado =
%.2f\n", x, x_norm);
    }
    return 0;
```

QUESTÃO 3 - CÓDIGO CLAUDE

```
#include <stdio.h>
#include <stdint.h>
int main(void) {
    unsigned long n, i;
    uint8_t pixel, x;
    int xmin, xmax;
    double x_norm;
    printf("Quantidade de pixels (n >= 1): ");
    if (scanf("%lu", &n) != 1 || n < 1) {
        fprintf(stderr, "Entrada invalida.\n");
        return 1;
    }
    printf("Digite a intensidade do pixel 1: ");
    if (scanf("%hhu", &pixel) != 1) {
        fprintf(stderr, "Entrada invalida.\n");
        return 1;
    }
    xmin = xmax = pixel;
    for (i = 2; i <= n; i++) {
        printf("Digite a intensidade do pixel %lu:
", i);
        if (scanf("%hhu", &pixel) != 1) {
            fprintf(stderr, "Entrada invalida.\n");
            return 1;
        }
        if (pixel < xmin) xmin = pixel;
        if (pixel > xmax) xmax = pixel;
    }
    printf("xmin = %d, xmax = %d\n", xmin,
xmax);
    printf("Digite a intensidade x a ser
normalizada: ");
    if (scanf("%hhu", &x) != 1) {
        fprintf(stderr, "Entrada invalida.\n");
        return 1;
```

```
}
```

```
}
```

```
if (xmax == xmin) {  
    printf("Nao e possivel normalizar:  
    intervalo degenerado (xmax = xmin).\n");  
    return 0;  
}
```

```
x_norm = (double)((int)x - xmin) /  
(double)(xmax - xmin);  
printf("x' = %.4f\n", x_norm);
```

```
return 0;  
}
```

i) i) O Claude elevou as representações dos números alterando os tipos das variáveis. O domínio das variáveis é algo a se analisar no caso concreto. A alteração feita pelo Claude não influenciou minha metodologia.

ii) O Claude institui trava de entradas inválidas em todos os comandos de entrada. Essa prática será adotada a partir de agora.

iii) Meu algoritmo forçou valores ao inicializar as variáveis xmin e xmax. Desconfio que essa não é uma boa prática. O Claude iguala xmin e xmax à primeira entrada de pixel, depois, entrada a entrada, faz as comparações a cada laço e atualiza o valor de xmin e xmax, se for o caso. Essa parece ser uma boa prática. Por conta disso a primeira entrada fica fora do laço, que processa da segunda entrada até a entrada N.

iv) Creio que o Claude deveria ter analisado a igualdade de xmin e xmax antes de solicitar o valor do pixel a ser normalizado, evitando processamento desnecessário e incômodo para o usuário.

QUESTÃO 4 - CÓDIGO ALUNO LÍVIO SOUSA

```
#include<stdio.h>
int main(){
int numero, soma=0, contador=0;
do{
scanf("%d", &numero);
if(numero== -1){
printf("Soma = %d\n", soma);
printf("Quantidade = %d\n", contador);
}
if(numero>=0 && numero<=255){
soma += numero;
contador++;
}
}while(numero != -1);
return 0;
}
```

QUESTÃO 4 - CÓDIGO CLAUDE

```
#include <stdio.h>

int main(void) {
int numero, soma = 0, contador = 0;
int continuar = 1;

while (continuar) {
if (scanf("%d", &numero) != 1) {
fprintf(stderr, "Entrada invalida.\n");
return 1;
}

if (numero == -1) {
continuar = 0;
} else if (numero >= 0 && numero <= 255)
{
soma += numero;
contador++;
}
/* numero < 0 (diferente de -1) ou
numero > 255: descartado, sem acao */
}

printf("Soma = %d\n", soma);
printf("Quantidade = %d\n", contador);

return 0;
}
```

i) Dessa vez enviei para o Claude somente o comando da questão, sem enviar meu algoritmo ainda. Ele me retornou uma análise assintótica. Embora eu tenha tido dificuldade em compreender por completo a análise, achei interessante esse retorno. Já deu para ter uma noção de quais más práticas podem levar a comportamentos assintóticos indesejados.

ii) Segundo o próprio Claude, meu algoritmo está melhor estruturado melhor, pois enquanto

ele precisou criar uma variável de controle porque optou por um laço while, eu controlei a saída através do seu próprio valor usando um do-while. Segue transcrição da resposta do Claude: *"Onde sua solução é genuinamente melhor que a minha*

Você usou do-while com a própria condição do sentinela como condição de parada (while(numero != -1)), sem precisar de nenhuma variável de controle extra. Isso é mais enxuto que a minha versão, que introduziu uma variável continuar só para emular o break. Sua abordagem elimina um estado a mais no programa – e "menos estado para rastrear" é, de fato, um critério real de qualidade de código. Nesse ponto específico, sua solução é mais elegante que a minha."

iii) Criticou a saída dentro do laço, pois se os prints fossem para o final do código ficaria mais claro ao leitor o seu objetivo. Concordo com ele.

iv) Também criticou a ausência de validações de entrada, assunto discutido nas questões anteriores.

